

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет по лабораторной работе №3
по дисциплине «Сети ЭВМ и телекоммуникации
компьютерных сетей»
Вариант 9

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Мулюха А.В.

«_____» _____ 2026г.

Санкт-Петербург, 2026

1 Постановка задачи

Используя docker создать контейнеры, необходимые для реализации следующего функционала с использованием RabbitMQ, а также показать, как именно осуществляется передача в этих условиях.

Вариант 9. Организовать общение трех отправителей и трех получателей, при этом каждому получателю соответствует своя очередь. А отправители могут писать в каждую из очередей, используя механизм Header Exchange с различными комбинациями полей.

1.1 RabbitMQ

RabbitMQ — это транспортный слой для обмена сообщениями. Принцип работы:

1. Producer отправляет сообщение в Exchange.
2. Exchange принимает сообщение и в зависимости от его атрибутов отвечает за роутинг и дальнейший процессинг.
3. С помощью Bindings сообщение отправляется на нужную очередь (Queue).
4. Сообщение будет находится в Queue до тех пор, пока не будет обработано получателем (Consumer).
5. Consumer обрабатывает сообщение, подключившись к очереди (push модель).

Header Exchange поддерживает binding (связь между очередью и обработчиком сообщений) по заголовкам, то есть пары `key = value`. С помощью `x-match` можно задать стратегию соответствия:

- All - все пары должны совпасть.
- Any - хотя бы одна пара должна совпасть.

2 Docker

2.1 Установка Docker

Docker — это платформа для контейнеризации, которая позволяет упаковать приложение со всем его окружением и зависимостями (биб-

лиотеки, настройки) в изолированный «контейнер». Контейнеры обеспечивают переносимость приложения между различными средами.

1. С сайта <https://www.docker.com/products/docker-desktop/> скачать Docker Desktop для ОС, установленной на компьютере.
2. Установить и запустить Docker Desktop.

2.2 Docker Compose

Docker Compose — это инструмент для определения и запуска многоконтейнерных приложений Docker. С помощью файла `docker-compose.yml` можно описать инфраструктуру приложения (сервисы, переменные окружения) и запустить её одной командой.

Листинг 1. `docker-compose.yml`

```
1  version: "3.8"
2
3  services:
4  # ===== RABBITMQ =====
5  rabbitmq:
6  image: rabbitmq:3.13-management-alpine # Образ RabbitMQ с вебинтерфейсом-
   на Alpine Linux
7  container_name: rabbitmq_headers # Имя контейнера
8  ports:
9  - "5672:5672" # AMQP protocol
10 - "15672:15672" # Management UI
11 environment:
12 RABBITMQ_DEFAULT_USER: guest
13 RABBITMQ_DEFAULT_PASS: guest
14 RABBITMQ_LOG_LEVEL: error
15 networks:
16 - rabbitnet # Подключение к внутренней сети для взаимодействия сервисов
17 healthcheck:
18 test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
19 interval: 10s
20 timeout: 5s
21 retries: 5
22
23 # Каждый получатель слушает свою очередь с уникальной привязкой по заголовкам
24
25 alice_receiver:
26 build:
27 context: ./receiver # Путь к Dockerfile для сборки образа
28 dockerfile: Dockerfile
29 container_name: receiver_alice
30 environment:
31 AMQP_URL: amqp://guest:guest@rabbitmq:5672/ # URL подключения к RabbitM
32 QUEUE_NAME: queue_alice
33 RECEIVER_NAME: Alice
34 BINDING_HEADERS: '{"x-match": "any", "category": "tech"}'
35 networks:
```

```

36 - rabbitnet
37 depends_on:
38 rabbitmq:
39 condition: service_healthy
40 restart: on-failure
41
42 bob_receiver:
43 build:
44 context: ./receiver
45 dockerfile: Dockerfile
46 container_name: receiver_bob
47 environment:
48 AMQP_URL: amqp://guest:guest@rabbitmq:5672/
49 QUEUE_NAME: queue_bob
50 RECEIVER_NAME: Bob
51 BINDING_HEADERS: '{"x-match": "any", "category": "news"}'
52 networks:
53 - rabbitnet
54 depends_on:
55 rabbitmq:
56 condition: service_healthy
57 restart: on-failure
58
59 eva_receiver:
60 build:
61 context: ./receiver
62 dockerfile: Dockerfile
63 container_name: receiver_eva
64 environment:
65 AMQP_URL: amqp://guest:guest@rabbitmq:5672/
66 QUEUE_NAME: queue_eva
67 RECEIVER_NAME: Eva
68 BINDING_HEADERS: '{"x-match": "any", "level": "senior"}'
69 networks:
70 - rabbitnet
71 depends_on:
72 rabbitmq:
73 condition: service_healthy
74 restart: on-failure
75
76 # Каждый отправитель публикует сообщения с разными комбинациями заголовков
77
78 fred:
79 build:
80 context: ./sender
81 dockerfile: Dockerfile
82 container_name: sender_fred
83 environment:
84 AMQP_URL: amqp://guest:guest@rabbitmq:5672/
85 SENDER_NAME: Fred
86 MESSAGE_HEADERS: '["category": "tech", "level": "senior"]'
87 networks:
88 - rabbitnet
89 depends_on:
90 rabbitmq:
91 condition: service_healthy

```

```

92 restart: "no" # Отправитель завершается после отправки
93
94 simon:
95 build:
96 context: ./sender
97 dockerfile: Dockerfile
98 container_name: sender_simon
99 environment:
100 AMQP_URL: amqp://guest:guest@rabbitmq:5672/
101 SENDER_NAME: Simon
102 MESSAGE_HEADERS: '[{"category": "news", "level": "senior"}]',
103 networks:
104 - rabbitnet
105 depends_on:
106 rabbitmq:
107 condition: service_healthy
108 restart: "no"
109
110 trevor:
111 build:
112 context: ./sender
113 dockerfile: Dockerfile
114 container_name: sender_trevor
115 environment:
116 AMQP_URL: amqp://guest:guest@rabbitmq:5672/
117 SENDER_NAME: Trevor
118 MESSAGE_HEADERS: '[{"category": "tech", "category_alt": "news"}]',
119 networks:
120 - rabbitnet
121 depends_on:
122 rabbitmq:
123 condition: service_healthy
124 restart: "no"
125
126 networks:
127 rabbitnet:
128 driver: bridge

```

2.3 DockerFile

Dockerfile — это текстовый файл с инструкциями для сборки Docker-образа. Каждая инструкция создаёт новый слой образа.

Листинг 2. DockerFile

```

1 FROM python:3.10-slim
2
3 # Установка рабочей директории внутри контейнера
4 WORKDIR /app
5
6 # Обновляем CA сертификаты и pip
7 RUN apt-get update && apt-get install -y --no-install-recommends ca-
   certificates && \
8 pip install --upgrade pip setuptools && \

```

```

9 apt-get clean && rm -rf /var/lib/apt/lists/*
10
11 # Переменная для вывода логов в реальном времени
12 ENV PYTHONUNBUFFERED=1
13
14 COPY ../requirements.txt .
15 RUN pip install --no-cache-dir \
16 --index-url https://pypi.tuna.tsinghua.edu.cn/simple \
17 -r requirements.txt
18 # Копирование исходного кода приложения в образ
19 COPY . .
20 # Команда по умолчанию при запуске контейнера:
21 CMD ["python", "-u", "receiver.py"]

```

Для отправителя нужно поменять строку CMD ["python u "receiver.py"] на CMD ["python u "sender.py"]

Команды для сборки и запуска:

- Сборка образов и запуск всех сервисов `docker-compose up -build`
- Просмотр логов в реальном времени `docker-compose logs -f`
- Остановка и удаление контейнеров `docker-compose down`

3 Отправитель и получатель

Механизм работы:

- Отправитель (sender) подключается к RabbitMQ и публикует сообщения в обменник Header Exchange с определенным набором пользовательских заголовков.
- Exchange получает сообщение, извлекает из него заголовки и проверяет, какому условию x-match оно соответствует. Если сообщение подходит условию, оно копируется в соответствующую очередь. Одно сообщение может скопироваться в несколько очередей одновременно.
- Получатель (receiver) подписывается на свою очередь. При поступлении сообщения вызывается коллбэк, а после обработки сообщения оно удаляется из очереди.

Листинг 3. receiver.py

```

1 #!/usr/bin/env python3
2 import os
3 import sys

```

```

4 import json
5 import time
6 import pika
7
8 # Чтение настроек из переменных окружения
9 AMQP_URL = os.getenv('AMQP_URL', 'amqp://guest:guest@localhost:5672/')
10 QUEUE_NAME = os.getenv('QUEUE_NAME', 'default_queue')
11 RECEIVER_NAME = os.getenv('RECEIVER_NAME', QUEUE_NAME)
12 BINDING_HEADERS_RAW = os.getenv('BINDING_HEADERS', '{"x-match": "any"}')
13 BINDING_HEADERS = json.loads(BINDING_HEADERS_RAW)
14
15 EXCHANGE_NAME = 'headers_exchange'
16
17 def connect_to_rabbitmq():
18     """Подключение к RabbitMQ с повторными попытками."""
19     max_attempts = 30
20     for attempt in range(max_attempts):
21         try:
22             connection = pika.BlockingConnection(
23                 pika.URLParameters(AMQP_URL)
24             )
25             print(f"[{RECEIVER_NAME}] Подключено к RabbitMQ")
26             return connection
27         except pika.exceptions.AMQPConnectionError:
28             time.sleep(2)
29     raise RuntimeError(f"[{RECEIVER_NAME}] Не удалось подключиться к RabbitMQ")
30
31
32 def on_message(ch, method, properties, body):
33     """Обработчик полученного сообщения."""
34     msg = body.decode()
35     headers = getattr(properties, 'headers', {})
36     print(f"[{RECEIVER_NAME}] Получено: '{msg}'. Заголовки сообщения: {headers}"
37         )
38     print("-" * 60)
39     # Подтверждаем получение
40     ch.basic_ack(delivery_tag=method.delivery_tag)
41
42 def main():
43     connection = connect_to_rabbitmq()
44     channel = connection.channel()
45     # Настройка QoS: обрабатывать по 1 сообщению за раз
46     channel.basic_qos(prefetch_count=1)
47     # Объявляем очередь
48     channel.queue_declare(queue=QUEUE_NAME, durable=True)
49     print(f"[{RECEIVER_NAME}] Очередь '{QUEUE_NAME}' готова")
50     # Объявляем Exchange должен( совпадать с отправителем)
51     channel.exchange_declare(
52         exchange=EXCHANGE_NAME,
53         exchange_type='headers',
54         durable=True
55     )
56     # привязка очереди к Exchange с заголовками
57     channel.queue_bind(
58         exchange=EXCHANGE_NAME,

```

```

59 queue=QUEUE_NAME,
60 arguments=BINDING_HEADERS
61 )
62 print(f"[{RECEIVER_NAME}] Привязка: {BINDING_HEADERS}")
63 # Подписываемся на получение сообщений
64 channel.basic_consume(
65     queue=QUEUE_NAME,
66     on_message_callback=on_message,
67     auto_ack=False # ручное подтверждение
68 )
69 print(f"[{RECEIVER_NAME}] Ожидание сообщений... (Ctrl+C для выхода)")
70 try:
71     channel.start_consuming()
72 except KeyboardInterrupt:
73     print(f"\n[{RECEIVER_NAME}] Остановка...")
74     channel.stop_consuming()
75     connection.close()
76 if __name__ == '__main__':
77     main()

```

Листинг 4. sender.py

```

1  #!/usr/bin/env python3
2  import os
3  import sys
4  import json
5  import time
6  import pika
7
8  # Чтение настроек из переменных окружения
9  AMQP_URL = os.getenv('AMQP_URL', 'amqp://guest:guest@localhost:5672/')
10 SENDER_NAME = os.getenv('SENDER_NAME', 'Unknown')
11 # MESSAGE_HEADERS: JSONмассив-словарей заголовков для каждого сообщения
12 MESSAGE_HEADERS_RAW = os.getenv('MESSAGE_HEADERS', '[]')
13 MESSAGE_HEADERS = json.loads(MESSAGE_HEADERS_RAW)
14 EXCHANGE_NAME = 'headers_exchange'
15 EXCHANGE_TYPE = 'headers'
16
17 def connect_to_rabbitmq():
18     """Подключение к RabbitMQ с повторными попытками."""
19     max_attempts = 30
20     for attempt in range(max_attempts):
21         try:
22             connection = pika.BlockingConnection(
23                 pika.URLParameters(AMQP_URL)
24             )
25             print(f"[{SENDER_NAME}] Подключено к RabbitMQ")
26             return connection
27         except pika.exceptions.AMQPConnectionError:
28             time.sleep(2)
29     raise RuntimeError(f"[{SENDER_NAME}] Не удалось подключиться к RabbitMQ")
30
31 def main():
32     connection = connect_to_rabbitmq()
33     channel = connection.channel()
34     # Объявляем Header Exchange

```

```

35 channel.exchange_declare(
36     exchange=EXCHANGE_NAME,
37     exchange_type=EXCHANGE_TYPE,
38     durable=True
39 )
40 print(f"[{SENDER_NAME}] Exchange '{EXCHANGE_NAME}' готов")
41 # Отправляем сообщения с разными заголовками
42 for idx, headers in enumerate(MESSAGE_HEADERS, 1):
43     body = f"Message #{idx} from {SENDER_NAME}"
44     # x-match указывается только в аргументах привязки очереди
45     properties = pika.BasicProperties(
46         delivery_mode=2,
47         content_type='text/plain',
48         headers=headers
49     )
50     channel.basic_publish(
51         exchange=EXCHANGE_NAME,
52         routing_key='', # игнорируется для header exchange
53         body=body.encode(),
54         properties=properties
55     )
56     print(f"[{SENDER_NAME}] Отправлено: '{body}' | Заголовки: {headers}")
57     time.sleep(1) # небольшая задержка между сообщениями
58     connection.close()
59     print(f"[{SENDER_NAME}] Завершено")
60
61 if __name__ == '__main__':
62     main()

```

4 Результаты работы

Лог запуска RabbitMQ:

```

rabbitmq_headers | ## ##      RabbitMQ 3.13.7
rabbitmq_headers | ## ##
rabbitmq_headers | #####      Copyright (c) 2007-2024 Broadcom Inc
and/or its subsidiaries
rabbitmq_headers | ##### ##
rabbitmq_headers | #####      Licensed under the MPL 2.0. Website:
https://rabbitmq.com
rabbitmq_headers |
rabbitmq_headers | Erlang:      26.2.5.16 [jit]
rabbitmq_headers | TLS Library: OpenSSL - OpenSSL 3.1.8 11 Feb 2025
rabbitmq_headers | Release series support status: see
https://www.rabbitmq.com/release-information
rabbitmq_headers |
rabbitmq_headers | Doc guides:  https://www.rabbitmq.com/docs

```

```

rabbitmq_headers | Support:      https://www.rabbitmq.com/docs/contact
rabbitmq_headers | Tutorials:   https://www.rabbitmq.com/tutorials
rabbitmq_headers | Monitoring: https://www.rabbitmq.com/docs/monitoring
rabbitmq_headers | Upgrading:   https://www.rabbitmq.com/docs/upgrading
rabbitmq_headers |
rabbitmq_headers | Logs: <stdout>
rabbitmq_headers |
rabbitmq_headers | Config file(s):
/etc/rabbitmq/conf.d/10-defaults.conf

```

Логи работы отправителей и получателей:

```

sender_simon      | [Simon] Подключено к RabbitMQ
sender_simon      | [Simon] Exchange 'headers_exchange' готов
sender_simon      | [Simon] Отправлено: 'Message #1 from Simon'
| Заголовки: {'category': 'news', 'level': 'senior'}

sender_trevor     | [Trevor] Подключено к RabbitMQ
sender_trevor     | [Trevor] Exchange 'headers_exchange' готов
sender_trevor     | [Trevor] Отправлено: 'Message #1 from Trevor'
| Заголовки: {'category': 'tech', 'category_alt': 'news'}

sender_fred       | [Fred] Подключено к RabbitMQ
sender_fred       | [Fred] Exchange 'headers_exchange' готов
sender_fred       | [Fred] Отправлено: 'Message #1 from Fred'
| Заголовки: {'category': 'tech', 'level': 'senior'}

receiver_eva      | [Eva] Подключено к RabbitMQ
receiver_eva      | [Eva] Очередь 'queue_eva' готова
receiver_eva      | [Eva] Привязка: {'x-match': 'any', 'level': 'senior'}
receiver_eva      | [Eva] Ожидание сообщений... (Ctrl+C для выхода)
receiver_eva      | [Eva] Получено: 'Message #1 from Simon'.
Заголовки сообщения: {'category': 'news', 'level': 'senior'}
receiver_eva      | [Eva] Получено: 'Message #1 from Fred'.
Заголовки сообщения: {'category': 'tech', 'level': 'senior'}

receiver_alice    | [Alice] Подключено к RabbitMQ
receiver_alice    | [Alice] Очередь 'queue_alice' готова
receiver_alice    | [Alice] Привязка: {'x-match': 'any',
'category': 'tech'}
receiver_alice    | [Alice] Ожидание сообщений... (Ctrl+C для выхода)

```

```

receiver_alice      | [Alice] Получено: 'Message #1 from Trevor'.
Заголовки сообщения: {'category': 'tech', 'category_alt': 'news'}
receiver_alice      | [Alice] Получено: 'Message #1 from Fred'.
Заголовки сообщения: {'category': 'tech', 'level': 'senior'}

receiver_bob        | [Bob] Подключено к RabbitMQ
receiver_bob        | [Bob] Очередь 'queue_bob' готова
receiver_bob        | [Bob] Привязка: {'x-match': 'any', 'category': 'news'}
receiver_bob        | [Bob] Ожидание сообщений... (Ctrl+C для выхода)
receiver_bob        | [Bob] Получено: 'Message #1 from Simon'.
Заголовки сообщения: {'category': 'news', 'level': 'senior'}

sender_simon        | [Simon] Завершено
sender_trevor       | [Trevor] Завершено
sender_fred         | [Fred] Завершено

```

5 Заключение

В ходе лабораторной работы были созданы docker контейнеры, которые с помощью очереди RabbitMQ передают сообщения, используя механизм Header Exchange. С помощью логов запуска показано, как именно осуществляется передача. Контейнеры связаны между собой через Docker Compose.